

IDS Alert Prioritization and Strengthening

Asad-ullah Khan, Vishal Madipadga, Skye Pekerti, Pratidhi Pratyasha

Abstract—Intrusion detection systems (IDS) are critical to network and application security, as they can alert personnel that an attack is happening to that it can be countered in an effective manner. However, a big problem with IDS is the number of false positives that they can produce. A system that is too sensitive can be a burden not only for the administration but also for the users of the service. Furthermore, too many false positives can actually mask the presence of real attacks and make it difficult for administrators to combat cyber attacks effectively.

Index Terms—Security, Intrusion, Detection, Machine Learning

I. INTRODUCTION

Many applications already use IDS to detect and prevent attacks. Many of these systems perform adequately for the job at hand. A very well known and common application of IDS is blocking logins from foreign countries, which services like Gmail [1] use to prevent unauthorized access preemptively.

II. PLAN

1. Initial network design

We plan to make use of neural networks as our classifiers. This initial work will be to determine the specific layer setup and architecture to use for the network. Existing research in this area will be used to plan our approach. We will use a supervised model, as our primary dataset (CSE-CIC-IDS2018) is labeled. The network type that we will focus on are deep neural networks, possibly using residuals [2] to allow us to create very deep networks.

2. Development and testing network

After determining a favorable architecture, we will train and test on the data available, and tune the network. If performance is not deemed adequate, we will take a step back and refine the architecture.

3. Final refinement and deliverable

Once a suitable network that trains and runs well is found, we can finalize the network and turn it into a product that can actively run on network traffic captured by tools such as CICFlowMeter.

III. ISSUES

1. Network training

Network training can be very resource intensive and difficult to do without proper resources. Fortunately, with recent advances in hardware and network design, times to train have been significantly reduced. Despite this, a powerful computer is still needed. There are two options at our disposal: Google Colab, a free to use GPU available Jupyter notebook, or a personal computer running a single RTX 2080 Ti.

2. Network architecture

Existing research in using neural networks for IDS typically have not used residual networks, so this will be a challenging task to complete

3. Dataset

One of the biggest problems we encountered was with the dataset we decided to use to test. We found that there were a lot of issues with the dataset and as a result, had to scrap some earlier tests and fix the data. Part of the problem is the python library we are using called scikit-learn. In order to use the library, the data had to be the right format so that it could be used. As we found out, scikit-learn does not allow for the use non-numeric features so we ended up having to filter out all non-numeric features such as Source IP address and Destination IP address. Most of the other problems we have faced are about the different models, figuring out which ones to use and once we choose, figuring out how they work and how to implement them.

IV. SCHEDULE

Sep 27 th	• Literature Review
Oct 4 th	• Project Proposal
Oct 10 th	• Investigate datasets
Oct 17 th	• Neural network architecture
Nov 1 st	• Midterm Report/Demo
Nov 8 th	• Test and validate approach
Nov 22 nd	• Refine and finalize network
Dec 6 th	• Final Report

Fig. 1. Timeline of project

V. RELATED WORK

1. Evaluation of Machine Learning Algorithms in IDS systems [3]

This paper evaluates the accuracy of 7 different machine learning algorithms in classifying packets as attack packets or normal packets. The 7 algorithms were, J48, Random Forests, Random Trees, Decision Tables, Multilayer Perceptron, Naive Bayes, and Bayes Network. The dataset used in this experiment was the KDD (Knowledge Discovery in Databases) intrusion dataset. The dataset included 21 types of attacks categorized into 4 groups, DOS, R2L, U2R, and PROBE. 148753 records from the dataset were used to train/create each model, 79% were DOS attacks, 19% was normal traffic, and 2% were other attacks. 60,000 randomly selected records were used to test the model. The performance metrics used to evaluate each of the algorithms, was the number of true positives, true negatives, false positives, and false negatives that were classified. The true positive rate was calculated for each of the classifiers. The Random Forest classifier had the highest True Positive rate with 93.1% while the Random Tree classifier had the lowest with 90.6%. The false positive and false negative rates were calculated. The Random Tree classifier had the highest false negative rate at 9.3%. The Decision Table had the lowest at 0.02%. For the false positive rate, the Decision Table had the highest rate at 7.3%.

2. An Application of Machine Learning to Network Intrusion Detection [4]

The authors of this paper apply techniques of machine learning in an attempt to improve an IDS,

reducing the need for human analysis. The techniques used for their research are genetic algorithms and decision trees. They specify that their use of genetic algorithms uses the source IP address, destination IP address, source IP port, destination IP port, and network protocol as their attributes. Their decision tree uses the same attributes but undergoes a different pruning route than the genetic algorithm. The techniques used are the most promising section of the research and we may be able to use some of their applications in our research.

3. Optimal IDS Sensor Placement and Alert Prioritization Using Attack Graphs [5]

The researchers of this paper detail a way to generate where to optimally use an IDS to sense intrusions into the system to reduce the amount of resources needed to detect intrusions. Listed is a way to use an attack graph, a topological graph of a network with the different attack vectors listed, to find the optimal positions to use an IDS. While this isn't fully focused on the alerts the IDS generates and how to prune out the false positives, reducing the sensors of an IDS can lead to a reduction in false-positives in areas where there are few attack points. However, if we use this technique there is some manual overhead to determine where the attack vectors of the system are. If we use this technique, it may be possible to automate this and have it work in conjunction with other techniques to reduce false positives.

4. An Approach to Verify, Identify and Prioritize IDS Alerts [6]

IDS often appear problematic because of the huge number of false-positive alerts generated which diminish the urgency of addressing alerts pertaining to potential threats. This paper proposes a way to classify alerts based on alert verification and history. Verification focuses on improving the quality of alerts by using vulnerability assessment while history works on keeping the most frequently-occurring interesting and non-interesting alerts. Alerts are classified into two superclasses: Interesting (Relevant) and Non-interesting (Non-relevant), and then further divided as per severity, frequency and the source of generation. The

experimental results showed that the classification detected 99% of the vulnerabilities and verifying before classifying reduced the load on the classifier by 35%. This paper gives us a view on how we could effectively classify alerts for further analysis.

5. Real-time classification of IDS alerts with data mining techniques [7]

IDS alerts often require human intervention for verifying if the alert generated is true or false-positive. This paper proposes a fully automatic data mining approach to distinguish important network IDS alerts from false-positives and those of low importance. A frequent itemset mining based method is used for checking alert logs and creating a classifier which does not require manual inspection. Most alerts are triggered by few signatures. The proposed algorithm classifies alerts as interesting or routine. It also rebuilds the classifier periodically to adapt to changing network environments so that new vulnerabilities can be detected. The algorithm also works on finding the endpoints for each signature. The experimental results suggested that interesting alert set was 5-100 times smaller than the original log and this method is as efficient as other data mining methods. This paper is useful for building a fully-automated approach for classifying IDS alerts.

6. Using Data Mining Algorithms for Developing a Model for Intrusion Detection System (IDS) [8]

This paper looks at using k-means clustering to help IDS systems reduce false positives and false negatives. The NSL-KDD dataset, a reduced version of the KDD dataset with redundant and duplicate records removed, was used in the study. There were 22 types of data used, each corresponding to a specific attack, with one of them being normal traffic. The clustering algorithm used was as follows. A dataset D, containing all the data types was used, k objects from the D were extracted and used as the center of a cluster. The objects were clustered based on the mean value of each of the clusters. Clusters of 11, 22, 44, 66, and 88 were used. From this, the normal detection rate, attack detection rate, efficiency rate, false positive rate, and false negative rate were calculated. Using k-means clustering led to a high false negative rate

and using a cluster of 22 (corresponding to the number of data types in the dataset) yielded the highest attack detection rate and highest normal detection rate.

7. A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection [9]

This paper discussed the progress of research in using machine learning to detect intrusions. It contains a comprehensive list of types of data to be analyzed, as well as the various attempted machine learning approaches to analyze them, including Bayesian networks, clustering, decision trees, evolutionary computation, and many more. Each one contains references to studies that used those approaches, their success rates, and overall quality of the approach. The end of the paper contains a list of observations by the author on the methods used, as well as recommendations for future research.

VI. RESULTS

1. Data Cleanup

The first approach to this issue was to clean up areas of the dataset. Unfortunately, the IDS 2018 dataset had many issues which required it to be thoroughly filtered before its use. Some of these issues were duplicate CSV headers, misspelled filenames, extraneous columns, NaN and Infinity in the dataset, and other issues related to how the data needed to be loaded for our purposes. Because of these issues, we are also supplying the HDF5 file of this dataset [10], which can be loaded into memory almost instantly, and also comes with minimums and maximums in order to normalize the data (it does not come normalized).

2. Neural Network

1) *Basic DNN*: Following the results of [11], we used the same model in that paper as a base. A brief summary of that model is as follows: 3 hidden layers of 64 units each, with an output layer using the sigmoid function as the activation, and all hidden layers using ReLU. The target function was the binary cross-entropy loss function. The input to the model were all the continuous rows concatenated with the categorical embeddings for each categorical feature. We used the same dimensions

as the paper: 15 dimensional embeddings for the destination port, and 3 dimensional embedding for the protocol. We also remove any columns that have

$$\min(C) = \max(C) \quad (1)$$

as these columns do not provide any useful information for use. This setup is typical for classification problems.

2) *Attention Model*: Given that our data has a temporal dependency, it seemed quite logical to look at networks that can tackle sequences instead of individual timesteps. We decided to use a context window of 8 timesteps, which were each fed into the *same* sub-network which consists of a 3 hidden layers, 32 units each. This produced an 8 dimensional context vector A , which was applied to the timesteps as follows:

$$\tanh(s * A - s/2) * I \quad (2)$$

where s is a "scale factor" hyperparameter which can exaggerate weights given to timesteps, and I is the original 8 timestep sequence. After this, the resulting vector was fed into the basic model specified earlier

3) *Reduced Feature Set*: The dataset we used had around 80 features. We hypothesized that using a smaller feature set would improve the performance of neural network model. Furthermore, we also believed a reduced feature set would yield a few more benefits.:

- Training our neural network may be less computationally expensive as there would less inputs to the network.
- There may be a possible performance improvement in classifiers that suffer from the curse of dimensionality such as K Nearest Neighbors.
- We would get to remove features that were useless (features that have no information gain or very low information gain)

In order to perform our feature reduction we made use of a Decision Tree. In a Decision Tree, the tree attempts to split the data into homogeneous groups. To split the data, it uses the features provided to it. It will split the data based on a particular value of the feature in the data. In order to pick the best feature to split the data on, it calculates something called information gain for each feature.

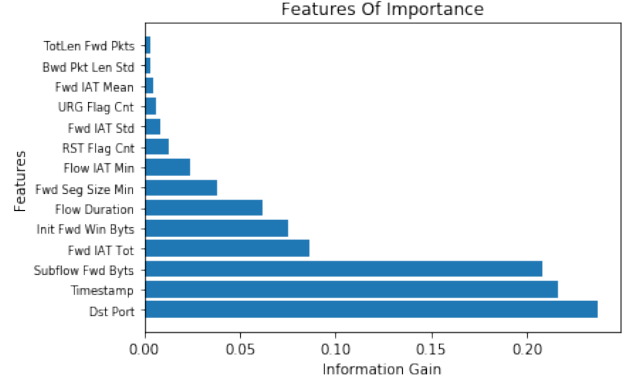


Fig. 2. The information gain for each feature, clipped to features with a gain of $> 10^{-3}$

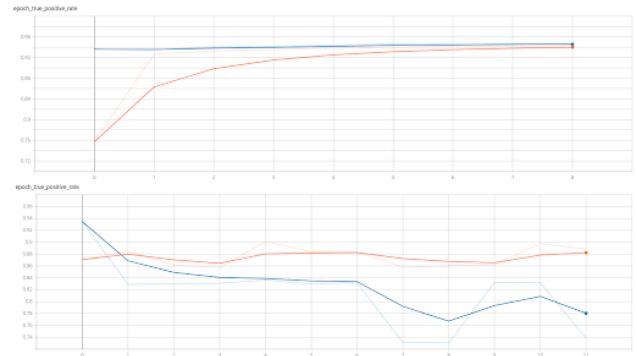


Fig. 3. Results of the attention network (top) and the basic neural network (bottom)

The feature with the highest information gain would be used to split the data.

After feeding the reduced features into the two networks created earlier, the results were much different than before. The attention model was able to generalize much better than the original DNN given the reduced feature set. This was surprising, given that they both exhibited similar performance given the full feature set. This suggests that the attention model generalizes better than the standard network. More testing and research needs to be done in this area to confirm this result

VII. CONCLUSION

Working off of the previous research, we solidified the merit in using different machine learning models in order to help create an accurate IDS. We improved upon the attention model, making it more accurate overall. Our attempt at creating a decision

tree also proved to be promising. Our experiment with the decision tree also allowed us to narrow down the set of features which were deemed most important. The smaller set of features could then be used to run other models at a faster rate since the number of calculations is greatly reduced with the reduced set of features.

REFERENCES

- [1] Google, "G suite admin help: Stop suspicious login activity alerts," 2019. [Online]. Available: <https://support.google.com/a/answer/7102416?hl=en>
- [2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *CoRR*, vol. abs/1512.03385, 2015. [Online]. Available: <http://arxiv.org/abs/1512.03385>
- [3] M. Almseidin, M. Alzubi, S. Kovacs, and M. Alkassabeh, "Evaluation of machine learning algorithms for intrusion detection system," in *2017 IEEE 15th International Symposium on Intelligent Systems and Informatics (SISY)*, Sep. 2017, pp. 000 277–000 282.
- [4] C. Sinclair, L. Pierce, and S. Matzner, "An application of machine learning to network intrusion detection," in *Proceedings 15th Annual Computer Security Applications Conference (ACSAC'99)*, Dec 1999, pp. 371–377.
- [5] S. Noel and S. Jajodia, "Optimal ids sensor placement and alert prioritization using attack graphs," *J. Network Syst. Manage.*, vol. 16, pp. 259–275, 09 2008.
- [6] T. H. Nguyen, J. W. Luo, and H. W. Njogu, "An approach to verify, identify and prioritize ids alerts," 2014.
- [7] R. Vaarandi, "Real-time classification of ids alerts with data mining techniques," in *MILCOM 2009 - 2009 IEEE Military Communications Conference*, Oct 2009, pp. 1–7.
- [8] S. Duque and M. Omar, "Using data mining algorithms for developing a model for intrusion detection system (ids)," *Procedia Computer Science*, vol. 61, pp. 46–51, 12 2015.
- [9] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys Tutorials*, vol. 18, no. 2, pp. 1153–1176, Secondquarter 2016.
- [10] A. ullah Khan, V. Madipadga, S. Pekerti, and P. Pratyasha, "Ids-2018 dataset combined into one hdf5 file," 2019. [Online]. Available: <https://drive.google.com/file/d/1GeCiqkdH3aqY8MUJztrvhHy-H0Ghd5Jo/view?usp=sharing>
- [11] G. C. Fernandez and S. Xu, "A case study on using deep learning for network intrusion detection," 2019.